

Applesoft's Loaded Dice

The Truth About Randomness on the Apple II, Then and Now

Jeff Robison · VCF Midwest 21 2026

The Problem

1

```
PRINT RND(1)  
.973136996
```

“...exactly the same sequence each time the machine is powered on.”
Aldridge, Behavior Research Methods, Instruments, & Computers, 1987

What a random number generator is

2

Truly random

Comes from something physical. A die. Thermal noise. You can't compute it.

Pseudo-random

A formula. You give it a starting number. It does math on that number and hands back an answer. The answer becomes the next starting number.

Entropy decides where the sequence starts. The formula decides what it looks like.

Integer BASIC, 1977

3

Woz wrote a generator at \$EF4E

- 15-bit shift register: repeats every 32,767 calls
- Integers only: RND(X) is the state MOD X
- Good enough for games, its stated purpose
- 56 bytes in all: 6 in KEYIN, 50 in RND

Seeded by you

There's a counter at \$4E and \$4F that goes up while the monitor sits waiting for a keypress. Integer BASIC's generator is that counter: RND scrambles it and writes it back.

Built-in Entropy Source

Same counter in the II, II Plus, IIe and IIc ROMs

<https://6502disassembly.com/a2-rom/OrigF8ROM.html>

```

fd1b: e6 4e      KEYIN      inc      RNDL
fd1d: d0 02                bne      KEYIN2      ;incr rnd number
fd1f: e6 4f                inc      RNDH
fd21: 2c 00 c0    KEYIN2    bit      IOADR      ;key down?
fd24: 10 f5                bpl      KEYIN      ; loop
fd26: 91 28                sta      (BASL),y    ;replace flashing screen
fd28: ad 00 c0                lda      IOADR      ;get keycode
fd2b: 2c 10 c0                bit      KBDSTRB    ;clr key strobe
fd2e: 60                                rts

```

<https://6502disassembly.com/a2-rom/AutoF8ROM.html>

```

fd1b: e6 4e      KEYIN      inc      RNDL
fd1d: d0 02                bne      KEYIN2
fd1f: e6 4f                inc      RNDH
fd21: 2c 00 c0    KEYIN2    bit      KBD          ;read keyboard
fd24: 10 f5                bpl      KEYIN
fd26: 91 28                sta      (BASL),y
fd28: ad 00 c0                lda      KBD
fd2b: 2c 10 c0                bit      KBDSTRB
fd2e: 60                                rts

```

https://6502disassembly.com/a2-rom/IIc_16kb.html

```

cc71: e6 4e                inc      RNDL      ;update seed
cc73: d0 1c                bne      UD2      ;check for key
cc75: a5 4f                lda      RNDH

```

https://6502disassembly.com/a2-rom/Unenh_IIe_80col.html

```

cb15: e6 4e      GETKEY    inc      MON_RNDL    ;bump random seed
cb17: d0 02                bne      GETK2
cb19: e6 4f                inc      MON_RNDH
cb1b: ad 00 c0    GETK2    lda      KBD          ;keypress?
cb1e: 10 f5                bpl      GETKEY    ;=>nope
cb20: 8d 10 c0                sta      KBDSTRB   ;clear strobe
cb23: 60                                rts

```

Integer BASIC, at \$EF4E

```

; Token $2f RND
; uses tken $3f (
;
ef4e: 20 15 e7    RND      jsr      GET16BIT
ef51: a5 4e      lda      MON_RNDL
ef53: 20 08 e7    jsr      LE708
ef56: a5 4f      lda      MON_RNDH
ef58: d0 04      bne      LEF5E
ef5a: c5 4e      cmp      MON_RNDL
ef5c: 69 00      adc      #$00
ef5e: 29 7f      and      #$7f
ef60: 85 4f      sta      MON_RNDH
ef62: 95 a0      sta      NOUNSTKC,x
ef64: a0 11      ldy      #$11
ef66: a5 4f      lda      MON_RNDH
ef68: 0a          asl      A
ef69: 18          clc
ef6a: 69 40      adc      #$40
ef6c: 0a          asl      A
ef6d: 26 4e      rol      MON_RNDL
ef6f: 26 4f      rol      MON_RNDH
ef71: 88          dey
ef72: d0 f2      bne      LEF66
ef74: a5 ce      lda      ACC
ef76: 20 08 e7    jsr      LE708
ef79: a5 cf      lda      ACC+1
ef7b: 95 a0      sta      NOUNSTKC,x
ef7d: 4c 7a e2    jmp      MOD

```

\$4E/\$4F



Applesoft II: Microsoft's RND

Microsoft's 6502 BASIC, licensed by Apple

```
*****
* RND statement
*****
efae: 20 82 eb  RND      jsr    SIGN      ;reduce argument to -1, 0, or +1
efb1: aa        tax      ;save argument
efb2: 30 18      bmi     LEFCC      ;= -1, use current argument for seed
efb4: a9 c9      lda     #RNDSEED    ;use current seed
efb6: a0 00      ldy     #>RNDSEED
efb8: 20 f9 ea  jsr     LOAD_FAC_FROM_YA
efbb: 8a        txa      ;recall sign of argument
efbc: f0 e7      beq     RTS_19      ;=0, return seed unchanged
efbe: a9 a6      lda     #<CON_RND_1 ;very poor RND algorithm
efc0: a0 ef      ldy     #>CON_RND_1
efc2: 20 7f e9  jsr     FMULT
efc5: a9 aa      lda     #<CON_RND_2 ;also, constants are truncated
efc7: a0 ef      ldy     #>CON_RND_2 ;<<< this does nothing, due to small exponent >>>
efc9: 20 be e7  jsr     FADD
efcc: a6 a1      LEFCC    ldx     FAC+4      ;shuffle hi and lo bytes
efce: a5 9e      lda     FAC+1      ;to supposedly make it more random
efd0: 85 a1      sta     FAC+4
efd2: 86 9e      stx     FAC+1
efd4: a9 00      lda     #$00      ;make it positive
efd6: 85 a2      sta     FAC_SIGN
efd8: a5 9d      lda     FAC        ;a somewhat random extension
efda: 85 ac      sta     FAC_EXTENSION
efdc: a9 80      lda     #$80      ;exponent to make value < 1.0
efde: 85 9d      sta     FAC
efe0: 20 2e e8  jsr     NORMALIZE_FAC_2
efe3: a2 c9      ldx     #RNDSEED    ;move FAC to RNDSEED
efe5: a0 00      ldy     #>RNDSEED
efe7: 4c 2b eb  GO_MVMF    jmp     STORE_FAC_AT_YX_ROUNDED
```

Seed at \$C9, from ROM

\$4E/\$4F still counts. Applesoft never reads it.

What it does

Takes the seed at \$C9, multiplies, adds a constant too small to matter, swaps the top and bottom bytes, forces the result under 1, writes it back to \$C9.

The Applesoft Manual

6

`RND (aexpr)`

Returns a random real number greater than or equal to 0 and less than 1.

`RND(n)`

If `\aexpr\` is greater than zero, `RND(aexpr)` generates a new random number each time it is used.

`RND(-n)`

If `\aexpr\` is less than zero, `RND(aexpr)` generates the same random number each time it is used with the same `\aexpr\`, as if from a permanent random number table built into the APPLE. If a particular negative argument is used to generate a random number, then subsequent random numbers generated with positive arguments will follow the same sequence each time. A different random sequence is initialized by each different negative argument. The primary reason for using a negative argument for `RND` is to initialize (or "seed") a repeatable sequence of random numbers. This is particularly helpful in debugging programs that use `RND`.

`RND(0)`

If `\aexpr\` is zero, `RND(aexpr)` returns the most recent previous random number generated (`CLEAR` and `NEW` do not affect this). Sometimes this is easier than assigning the last random number to a variable in order to save it.

.973136996

...starting here, every power-on.

**Repeatable on purpose:
documented.
Repeatable by accident:
not.**

Found, reported, worked around

7

Kaner & Vokey, MICRO 1984

Written 1982. RND fell into an endless loop of 202 numbers.

Call-A.P.P.L.E., Jan 1983

“RND is Fatally Flawed,” pp. 29–34.

Sander-Cederlof, AAL May 1984

Seed setup copies 4 of 5 bytes (fix: \$F151 \$1C→\$1D). Repetition starts at the 37,758th number.

Aldridge, BRMIC 1987

A memory experiment gave each day’s first subject the same “random” word list.

Gleason, 1988

Collegiate Microcomputer. Tested Apple IIe RND; suggested seeds that pass.

Empson, GS WorldView 1999

Write-up of Robert Moore's 1989 shift-register generator, seeded from \$4E/\$4F.

The Fix: Put the Counter Back

LC Loader.bin

1. Enables language card write mode while keeping ROM readable
2. Copies \$D000-\$FFFF (all of Applesoft and the monitor ROM) from ROM to LC RAM
3. Writes the patch code into the LC at \$F5CB, over HFIND (53 bytes), which no Applesoft routine calls.
4. Writes JMP \$F5CB at \$EFAE in the LC copy
5. Switches the language card to read mode

Patch_lc.bin

Keeps RND(-n) and RND(0) working as documented; positive arguments draw on the KEYIN counter.

Fixes the seed, not the generator. Needs a language card.

1977: Integer BASIC seeded RND from you. Then Applesoft seeded it from ROM.
It hid for years: rerun looked random; only power-on repeated.
The fix puts the counter back without changing a single program.

Try it yourself

[URL: Jeff to supply]

QR
Jeff to supply